

Introduction of an Application for Visualisation of Study Progression Analyses

Advanced R

Term Paper

Submitted to the Faculty of
Business Administration and Economics
at the
University of Duisburg-Essen

from:



Reviewer: Prof. Dr. Christoph Hanck

Deadline: April 7th 2020

Name:

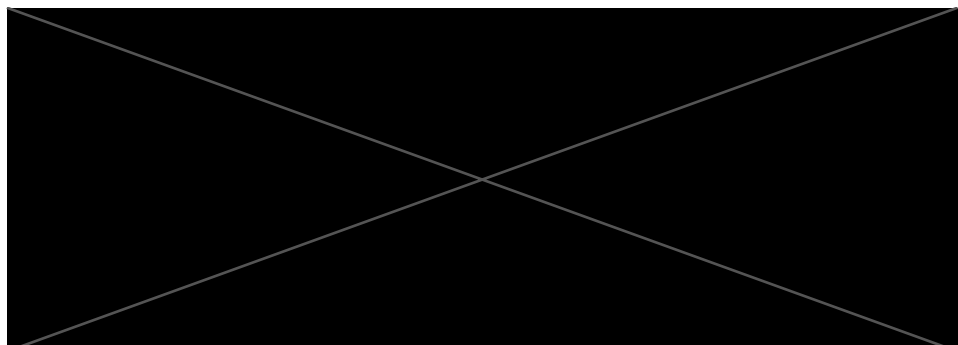
Matriculation No.:

E-Mail:

Study Path:

Semester:

Graduation (est.):



Contents

List of Figures	II
List of Tables	II
List of Abbreviations	II
1 Introduction	1
2 Study Progression Analysis	1
3 Data Structure	2
4 User Interface Design	2
4.1 FS Data	4
4.2 Pruefung Data	6
4.3 Studium Data	8
5 Special Features	9
5.1 Database Design	9
5.2 Persistent Filtering	10
5.3 Multilanguage Capability	12
6 Conclusion	13
References	15
Software-References	16
A Appendix	17
A.1 Description of relevant Variables	17

List of Figures

1	Selecting the Language of the Application.	3
2	Adding Data to the Data Base.	3
3	Overview of the Data Set.	4
4	Plot on Number of Cases per Exit Reason.	5
5	Plot on Number of Graduates per Semester.	5
6	Plot on Number of Exams per Semester.	6
7	Plot on Number of earned Credits per Semester.	7
8	Plot on Number of earned Credits per Semester.	7
9	Average Number of of required Attempts per Module.	8
10	Plot on Distribution of Academic Semesters in which Students have taken the Exam.	8
11	Plot on Distribution of Number of Students over the total Number of Semesters enrolled.	9

List of Tables

A1	Description of relevant variables	17
A1	Description of relevant variables (continued)	18

List of Abbreviations

ECTSCP	European Credit Transfer System Credit Point	1
---------------	--	---

1 Introduction

This paper presents a thorough description of the application for study progression analyses as well as its development process. The main focus of this application lies in the visualization of earned so-called European Credit Transfer System Credit Points (ECTSCPs) throughout students' studies. For this, we offer a user interface that makes it user-friendly to produce such graphics, which is described in more detail in the following paragraphs.

The remainder of this paper is structured as follows. Chapter 2 describes the relevance and benefits of study progression analyses and contains instructions for installing and starting the application. Chapter 3 gives an overview of the dataset and the necessary preprocessing. Chapter 4 describes the set up of the user interface. Chapter 5 presents the implemented features and gives insight into the core of the application. Chapter 6 concludes and suggests further extensions.

2 Study Progression Analysis

The world is constantly changing, and everyone has to adapt. This also affects companies that want to continue growing in the future. The goals of a university are, for example, to optimize the studying in general, but also to continually analyze and adapt the individual study programs. Study progress analyses are suitable as an analysis tool for this.

Data that reflect the course of studies of the individual students are considered as the basis. This includes, among other things, information on the respective examination regulations, examinations taken, length of studies, and reasons for dropping out (which also involves the completion of studies by graduation). These can be analyzed in different ways, so that general observations about the behavior of the students, but also study program or cohort related statements can be concluded.

Since the data on the individual students are somewhat similar, it makes sense to create an application that can be applied to many sets of historical study data and provides a basis for further analyses. Special attention is paid to the acquisition of ECTSCP by the students. Central questions are, for example, how many credit points a student acquires on average per semester or how the cumulative number of credits earned develops. Additionally, questions regarding the length of studies and the choice of subjects are taken into account. From these, conclusions can also be drawn about the offer of optional and compulsory subjects, e.g., about the level of difficulty.

We decided to publish the application as an R package on github.com. Installing the package is straightforward using a package like “remotes” or “devtools”. The following code utilizes the “remotes” package:

```
remotes::install_github("berrij/studydashboard")
```

The application itself was created using shiny and can be opened as follows:

```
studydashboard::launchApp()
```

Moreover, it is possible to open the application in the background using the “studydashboard::launchApp_background()” function as well as delete the stored data with the “reset_db()” function.

The package itself and all functions are documented and can be accessed using the common “?...” notation.

3 Data Structure

Based on the considerations described in Chapter 2, the data structure used for the study progression analyses is assumed to be the same for all study programs available.

The database for this application consists of three different datasets: “fs”, “pruefung”, and “studium”. The first one contains information on the general study situation of the students. This includes information on the status concerning the course of study, information on the specialization, or which examination regulations are in force for the student.

The second dataset “pruefung” contains information on the different exams for which a student has registered. Among others, there is information on the semester the exams took place, on the number of attempts, the degree, the earned ECTS, and whether the student has withdrawn from the exam.

The third dataset offers more detailed information on the students’ course of study. For each student, this information includes the semester in which she started and completed the study, information on the specialization, and the degree, among others. In total, this data set consists of 18 variables before and of 8 variables after removing those only consisting of missing values and those having no variation.

The most important variables are described in detail in annex A.1 in table A1.

4 User Interface Design

The structure of the app should be as intuitive as possible for the user so that even people with little computer or programming affinity should be able to use the tool without problems.

In order to keep possible barriers as low as possible, a dropdown menu on the left, middle edge of the screen allows users to switch between the languages English and German (see Figure 1). However, variables and their contents from the data sets cannot be changed easily, so “translation errors” may occur here.

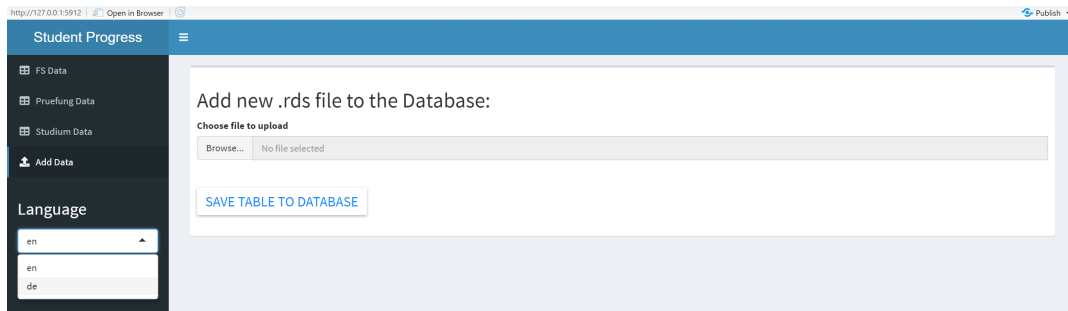


Figure 1: Selecting the Language of the Application.

The data sets can be uploaded above the language selection under the menu item “Add Data”, see Figure 2. As mentioned before, we have assumed during programming that the data sets always correspond to the three types described in chapter 3. The program ensures that only files of the type “.rds” can be uploaded. When selecting the “Browse...” button, a new window opens in which the file can be imported from the computer memory. Once this has been done, the user sees a preview of the selected data in the app and by clicking on “SAVE TABLE TO DATABASE” the table is added to the database within the program. This process can be repeated as often as required in order to read in all files to be examined with the app. The program automatically sorts the imported data tables into the three types “fs”, “pruefung” and “studium”.

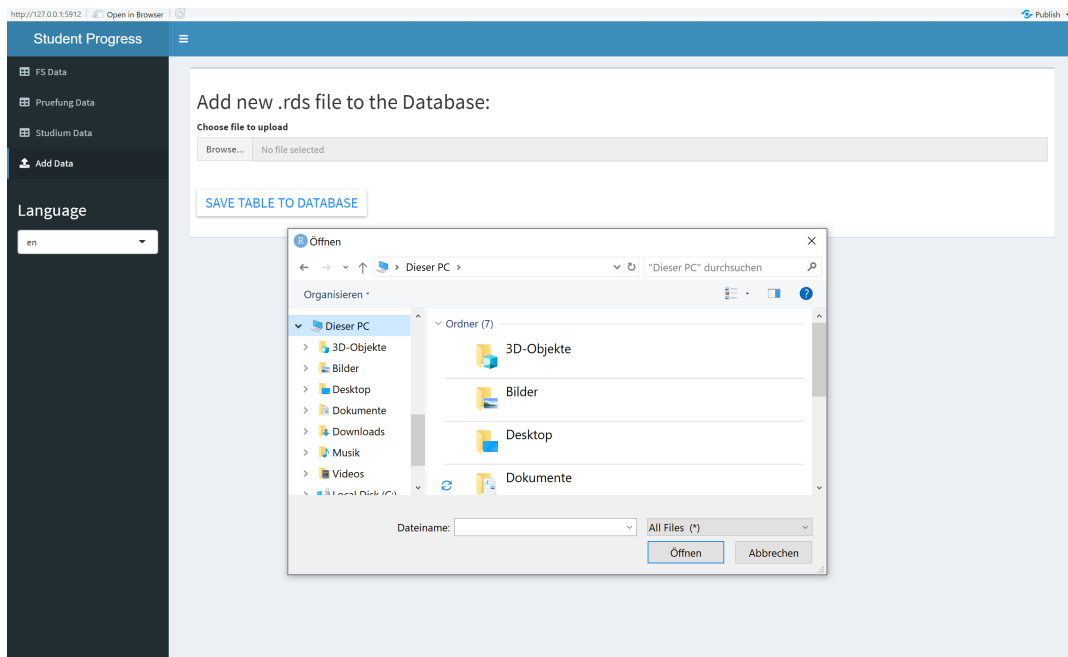


Figure 2: Adding Data to the Data Base.

In accordance with this distinction, the three further menu items were created. Opening one of these tabs allows the user to analyze the data. The main layout is the same for these three tabs. At the upper edge of the screen, the selected data type is displayed as a heading, for example, “FS Data” as can be seen in Figure 3. Directly below, there is a dropdown menu “Choose a dataset:” with which one can select the data set to be viewed in particular. It is important to

note that only those tables are available for selection that also belong to the corresponding type of data.

An overview of the data set with all variables and the first 10 observations is shown under the heading (see Figure 3). Directly below the variable names in the table, each variable can be filtered individually. For example, it is possible to filter for a single matriculation number and thus examine the study history data of an individual student. This is feasible because the filtering is saved and taken under consideration in the analyses below the table.

Open in Browser

Publish

Student Progress

FS Data

Pruefung Data

Studium Data

Add Data

Language

en

The FS Data:

Choose a dataset:

df_fs_MedBio

Show: 10 entries

Search:

Student.Pseudonym	Semester	FS	Status	Austrittsgrund	SGCode	SGDefinition_ID	Abschluss_Schlüssel	Abschluss_Bezeichnung	Fac
All				All		All	All	All	A
116151	20062	1	E		83(C01)-[H4]	17	83	Bachelor of Science (B. Sc.)	C01
116151	20071	2	R		83(C01)-[H4]	17	83	Bachelor of Science (B. Sc.)	C01
116151	20072	3	R		83(C01)-[H4]	17	83	Bachelor of Science (B. Sc.)	C01
116151	20081	4	R		83(C01)-[H4]	17	83	Bachelor of Science (B. Sc.)	C01
116151	20082	5	R		83(C01)-[H4]	17	83	Bachelor of Science (B. Sc.)	C01
116151	20091	6	R		83(C01)-[H4]	17	83	Bachelor of Science (B. Sc.)	C01
116772	20062	1	E		83(C01)-[H4]	17	83	Bachelor of Science (B. Sc.)	C01
116772	20071	2	R		83(C01)-[H4]	17	83	Bachelor of Science (B. Sc.)	C01
116772	20072	3	R		83(C01)-[H4]	17	83	Bachelor of Science (B. Sc.)	C01
116772	20081	4	R		83(C01)-[H4]	17	83	Bachelor of Science (B. Sc.)	C01

Figure 3: Overview of the Data Set.

The structure of each page is divided into boxes, which are lined up one below the other. The first box shows the table described above, followed by further boxes each containing an illustration. If the focus is to be placed on individual boxes, the boxes that are currently not relevant can be folded in using the “-” at the top right and unfolded again by clicking on “+”. At the bottom right corner of each box, there is an information button that gives the user more details on the respective figure, and in the lower left corner there is a download button with which the graphic can be saved as a PNG file.

The individual figures are described in more detail below, and some examples of their use and interpretation are given, starting with the menu item “FS Data”, followed by “Pruefung Data” and ending with “Studium Data”.

4.1 FS Data

The first chart (see Figure 4) in the tab concerning the FS data set is a bar chart for examining the reasons why students drop out. The absolute number of students is sorted and displayed according to the reasons for leaving the study program. For example, by selecting a different program for comparison above the graphic, the distributions of the reasons for dropping out can

be compared. Filtering the data sets, for instance, by examination regulations, also enables the analysis of dependencies.

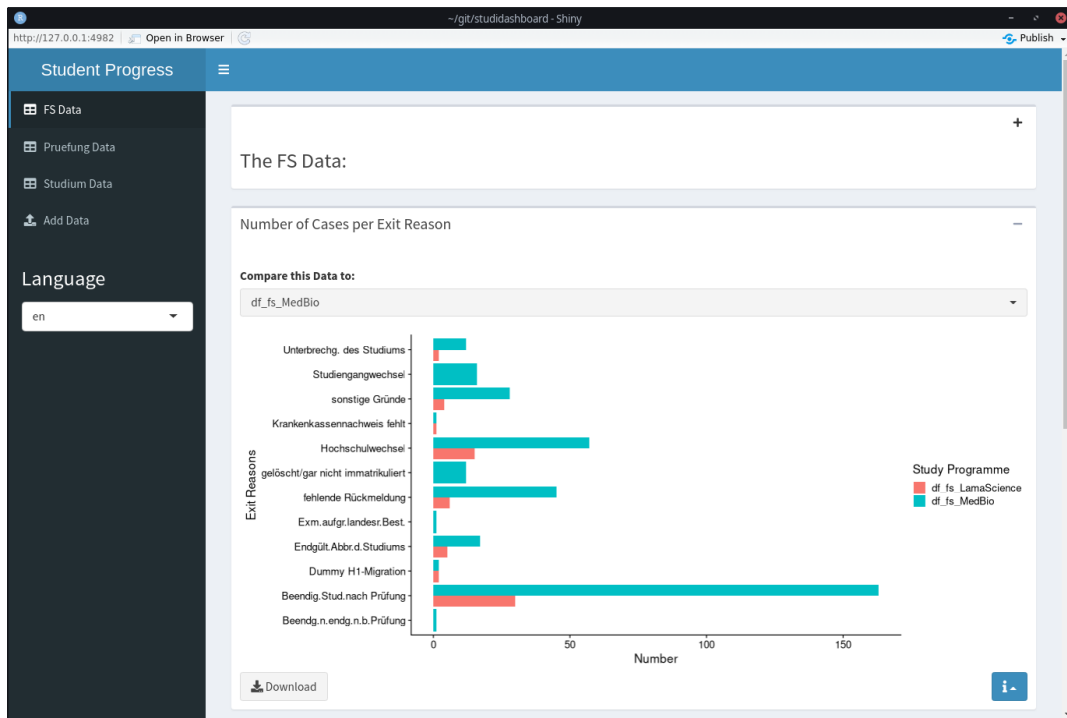


Figure 4: Plot on Number of Cases per Exit Reason.

The second plot (see Figure 5) shows the number of graduates per semester. To create this plot, only those students who have successfully completed their study program, i.e., those who have “Beendig.Stud.nach Prüfung” (Completed Studies after Examination) as the reason for dropping out, are selected. These observations are then sorted by the semester of graduation. In this way, patterns can be identified, such as more frequent graduations in the winter or summer semester, or whether there is a falling or rising tendency in the number of graduates.

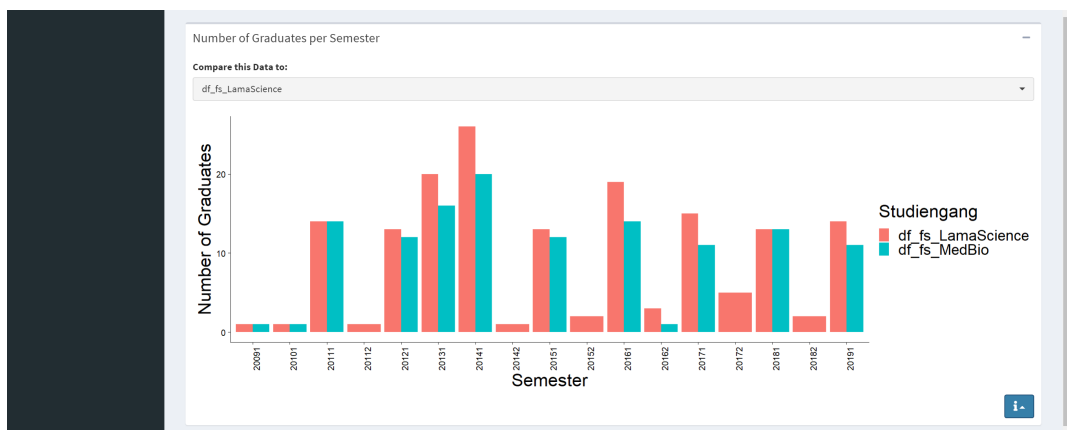


Figure 5: Plot on Number of Graduates per Semester.

4.2 Pruefung Data

The module at the top of the second tab, the Pruefung Data tab, is again a box with an overview of the data set with the possibility of setting filters for the data. Below this box, there are five more boxes with visualizations of relationships that might be of interest.

The first plot (see Figure 6), a bar chart, shows the number of examinations for which students have registered for each semester of examination. The semesters are displayed on the x-axis and the absolute number of assessments on the y-axis. By filtering the table above, for example, for the variable “Status.Pruefung” (Status of examination) by entering “B”, the system only displays the assessments which students have either passed (BE) or failed (NB). If a student were to withdraw from an assessment registration, he or she would still be included in the graphic at the beginning but would not be included after the filtering process.

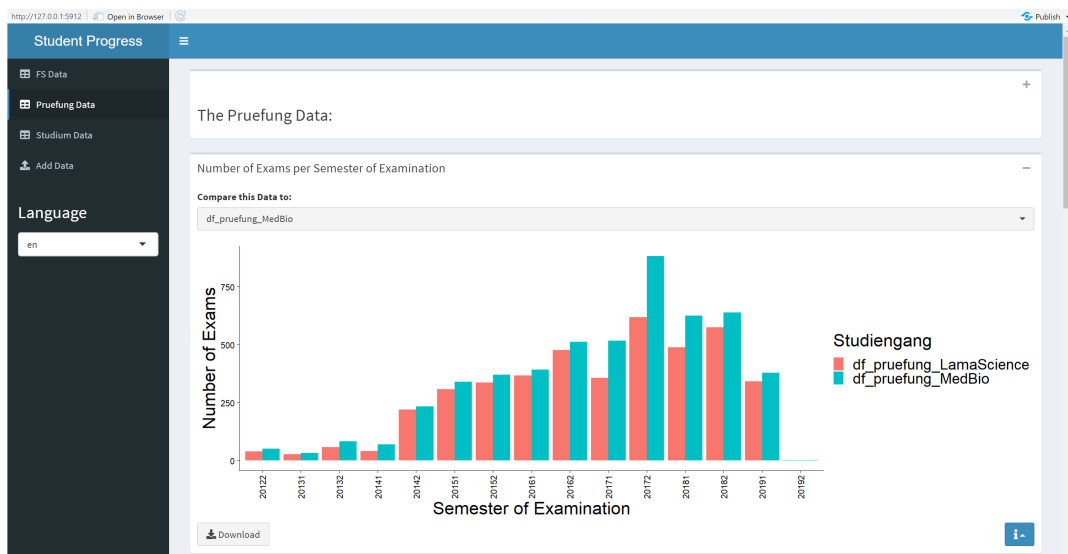


Figure 6: Plot on Number of Exams per Semester.

The second graph, see Figure 7, shows the average number of recorded ECTSCP at a cumulative level as the study progresses, i.e., as the number of academic semesters increases. For the calculation, the variable “verbuchteECTSCP” (recorded ECTSP) is first summed up for the individual semesters, then cumulated over the number of academic semesters and finally averaged.

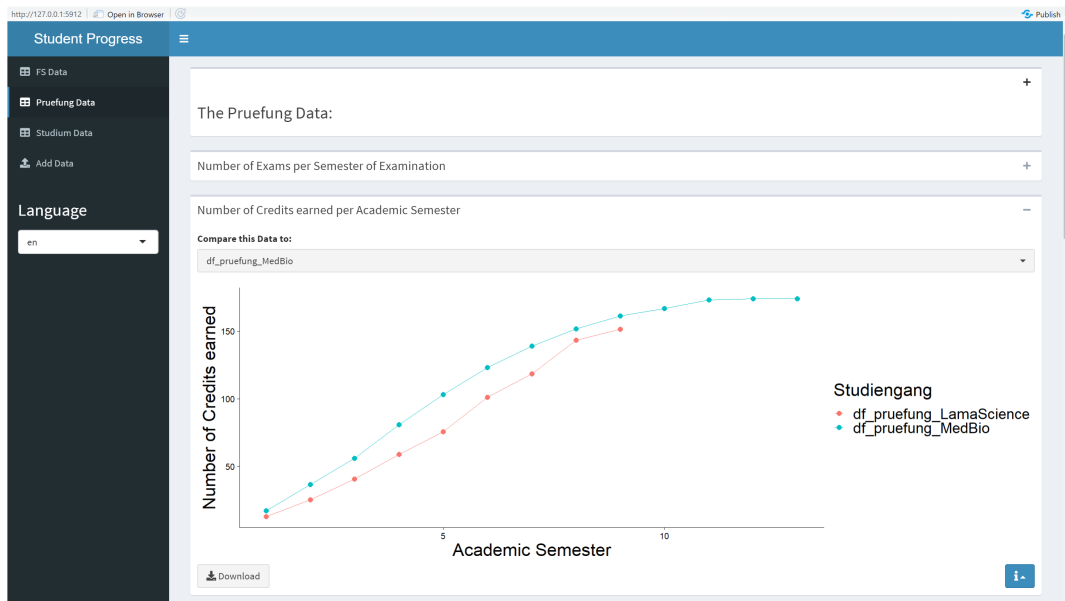


Figure 7: Plot on Number of earned Credits per Semester.

The third plot gives an overview of the average number of ECTS per examination semester. For this, the total number of gained ECTS per examination semester is divided by the number of students enrolled in that semester. Figure 8 shows the generated output.

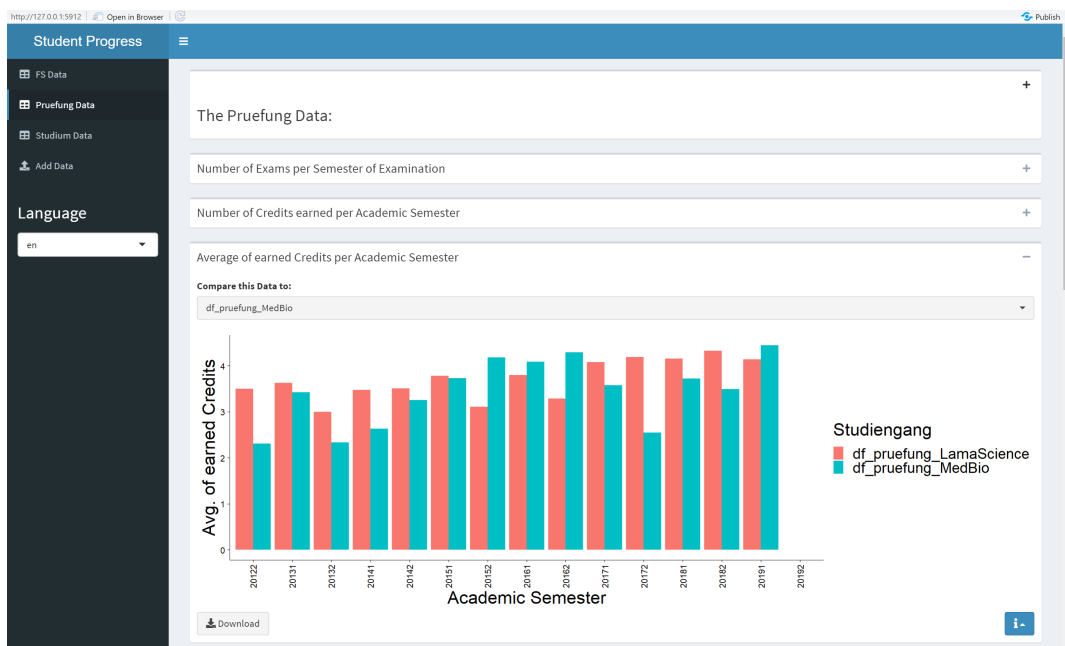


Figure 8: Plot on Number of earned Credits per Semester.

The fourth graph (see Figure 9) again is a bar chart showing the average number of required attempts per module. If the user does not filter the above table for students having passed the exam, the chart will also include students who have not passed the subject or who have finished/canceled their studies. The subjects to be compared can be selected using the selection

button located above the graph. By default, the first ten modules are selected based on the alphabetical order. When comparing two datasets, only those subjects with at least one observation in both data sets are available for selection. If a subject can be selected by students from different courses, this illustration can, for example, be used to compare which students need more or fewer attempts.

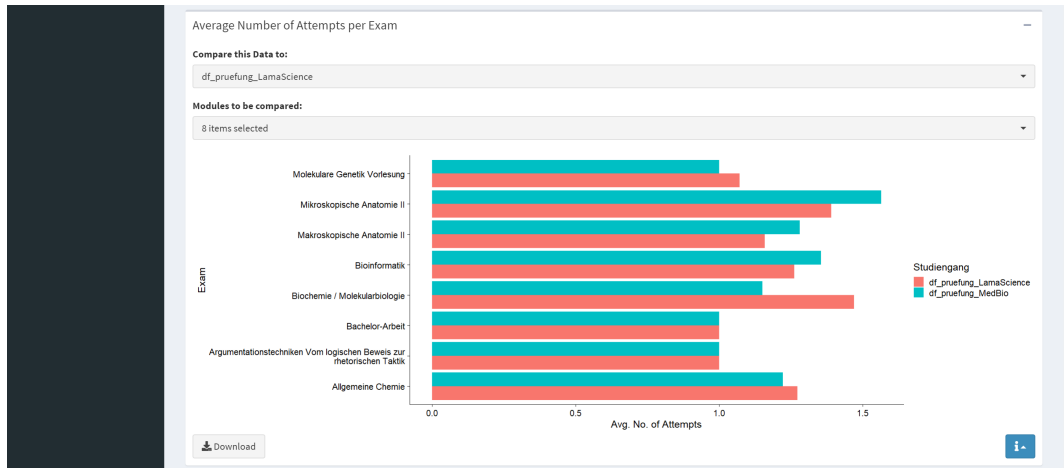


Figure 9: Average Number of of required Attempts per Module.

Figure 10 illustrates the last plot of this menu item and shows various bar charts of the absolute number of students who have registered for the examination for each module in a specific semester. This can be used, for example, to evaluate and optimize module plans based on empirical results.



Figure 10: Plot on Distribution of Academic Semesters in which Students have taken the Exam.

4.3 Studium Data

The bar chart in Figure 11 depicts how long the students were or are enrolled in the study program under consideration. The x-axis shows the absolute number of students and the y-axis the total number of semesters enrolled. For instance, if one adds a student to the dataset who is only enrolled in the program for one semester, the bar increases by a value at y equals 1.

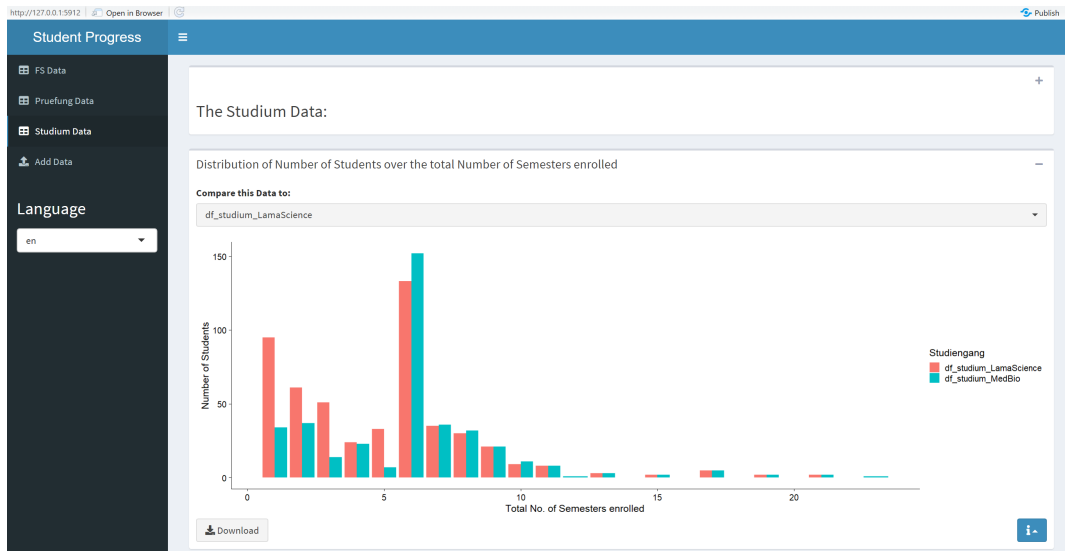


Figure 11: Plot on Distribution of Number of Students over the total Number of Semesters enrolled.

5 Special Features

5.1 Database Design

As this application’s primary purpose is exploratory data analysis, storing the data was one of the major objectives during development. Thereby, the solution had to fulfill various requirements. The user should be able to add datasets without being technically well educated. The data should be saved persistently, i.e., data that is added once by the user should be persistently saved and therefore reloaded after restarting the app. Lastly, since this application will be used and potentially extended by the university, the technical implementation should be easy to understand for people with basic knowledge of R programming.

Considering the aforementioned requirements, we decided to use three distinct SQL databases, each storing one type of data. This solution ensures that the tables in every database are homogenous. Moreover, implementing the storage solution using SQL warrants that third persons can easily extend or modify the database.

At each start of the application connections to the databases are initialized using the following code:

```
# Init db connections
fs <- pool::dbPool(drv = RSQLite::SQLite(), dbname = "data/fs.db")
pruefung <- pool::dbPool(drv = RSQLite::SQLite(), dbname = "data/pruefung.db")
studium <- pool::dbPool(drv = RSQLite::SQLite(), dbname = "data/studium.db")
```

We are using the “pool” package to handle the database connections. This is convenient because pool opens and closes connections to the database as needed.

The shiny application itself uses a reactive list object to store the database tables. Those objects are initialized as follows:

```
# Create a reactive object which stores all tables
data <- reactiveValues()
# Add FS Data from DB
data[['fs']] <- pool::dbListTables(fs) %>%
  purrr::map(.f = function(x) pool::dbReadTable(fs, x)) %>%
  setNames(pool::dbListTables(fs))
# Add Studium Data from DB
data[['studium']] <- pool::dbListTables(studium) %>%
  purrr::map(.f = function(x) pool::dbReadTable(studium, x)) %>%
  setNames(pool::dbListTables(studium))
# Add Pruefung Data from DB
data[['pruefung']] <- pool::dbListTables(pruefung) %>%
  purrr::map(.f = function(x) pool::dbReadTable(pruefung, x)) %>%
  setNames(pool::dbListTables(pruefung))
```

We chose to use a list because, on the one hand, list elements can be named, and on the other hand, lists can contain lists. That is, we create three lists inside “data” each named by the type of the tables it contains. The tables are then extracted out of each SQL database using the “map()” function of the purr packages. This function iterates over each table name, which is extracted upfront using dbListTables and extracts the respective tables. Lastly, the “setNames” function of the stats package is used to set the names of each element to its name inside the SQL database. This name equals the name of the file which the user has added to the database (without file extension).

Since “data” is a reactive object, elements of the shiny app change dynamically when changes to “data” are applied. This is the case when the user adds new data. We skip the code here as this would go beyond the scope of this documentation, but mainly, after adding the data to the database, the above code is executed again for the respective database to add the new database table to the reactive “data” object. Input objects relying on “data” will change immediately as “data” changes.

5.2 Persistent Filtering

Besides storing the data, filtering the data is another main feature of the application. We decided to use the “DT” (also known as datatable) package to present and filter the data. This is convenient because it allows the user to filter the datatable by an arbitrary variable without the need for creating input elements for each variable. Since we allow the user to switch between different tables (study programs), the chosen table is rendered each time the user switches between the tables. This creates a problem. Since the “renderDT” function does not remember which filters were applied when the data changes, it does not re-apply the filters when the user

reopens the same table at a later time. This behavior is not intuitive, and therefore we created a workaround.

Persistent filtering is achieved as follows. First, in addition to the table itself, a proxy object is created. Those proxy objects allow applying the filters to the respective data table. Moreover, we create a reactive list which stores the filters of each table as well as a reactive vector which stores the name of the table that is currently shown. The following code shows this initialization process for the FS data:

```
dt_colfilters <- reactiveValues()
dt_colfilters[['fs']] <- list()
last_opened_fs <- reactiveVal()
proxy_fs <- DT::dataTableProxy("table_fs")
```

The “fs” table is then created as follows:

```
# Observe if user selects a different fs table
observeEvent(input$dataset_fs, {
  # Init the variable at session start
  if(is.null(last_opened_fs())){
    last_opened_fs(input$dataset_fs)
    dt_colfilters[['fs']] [[last_opened_fs()]] <- NULL
  }
  # On input$dataset_fs change: store dt filters from the old dt
  dt_colfilters[['fs']] [[last_opened_fs()]] <- input$table_fs_search_columns
  # Render the new dt
  output$table_fs <- DT::renderDT(pool::dbReadTable(fs, input$dataset_fs),
                                   filter = "top",
                                   rownames = FALSE,
                                   options = list(scrollX = TRUE))
  # Apply the filters which are stored in dt_colfilters
  proxy_fs %>% DT::updateSearch(
    keywords = list(columns = dt_colfilters[['fs']] [[input$dataset_fs]])
  )
  # Save which table is opened
  last_opened_fs(input$dataset_fs)
})
```

This means, when the “last_opened_fs” object is empty, no table was rendered before (for the fs data). In that case, “last_opened_fs” is set to the current user selection, and the respective entry of the dt_colfilters[“fs”] reactive is set to NULL, meaning that no filters were applied. This step is only executed on initialization. When the user switches from one table to another the current filter options are extracted using “input\$table_fs_search_columns” and saved into the respective “dt_colfilters[['fs']] [[last_opened_fs()]]” object. Then, the new table is rendered,

and afterward, the proxy object is used to apply any filter options that might be present in the “dt_colfilters[['fs']]” entry which corresponds to the newly chosen table. Lastly, the new table name is assigned to the “last_opened_fs” object, which will be used to save the filter options if the user changes the table again.

This solution ensures that looking at different tables does not cause the filters to be lost.

We did create a similar reactive list-based approach to save the row indices of each table, which are left after the filters were applied. The code for this is presented below.

```
observe({
  if(nchar(input$dataset_fs) != 0){
    if(!is.null(input$table_fs_rows_all)){
      index_list[['fs']] [[input$dataset_fs]] <- input$table_fs_rows_all
    }else{
      index_list[['fs']] [[input$dataset_fs]] <-
        1:nrow(pool::dbReadTable(fs, input$dataset_fs))
    }
  }
})
```

The first condition simply checks whether an “fs” dataset is selected by the user. Which won’t be the case if the fs database is empty. If this condition holds, the “input\$table_fs_rows_all” object is evaluated. It stores the currently selected rows if the user has applied any filters and NULL otherwise. In consequence, we are storing all row numbers of the table in the index_list object if the “input\$table_fs_rows_all” is “NULL” or the specific rows if the filter were applied.

The index_list object is crucial for the application because it is used to filter the data before any plot is produced. That means the index_list object ensures that all filters that are set by the user will take effect on the plots immediately.

5.3 Multilanguage Capability

Since this application is part of an exam of the English module Advanced R, it was initially programmed in English. However, because it is based on German data and is designed to be used by a German-speaking team, it was essential making the app accessible to a multilingual group.

For implementation of this feature the “shiny.i18n” package was used. The basis of this set up is a so-called “.json” (JavaScript Object Notation) file. This file contains the translations of all strings into all languages which should be available. As the primary language of the code is English, English is also set as the primary language for the translation set up, as well as the language in which the app is started

The first part of implementing the translation is to set up a translator object loading the aforementioned .json file.

```
# Translation Set Up
translator <- Translator$new(translation_json_path = "data/translation_de.json")
```

This translator object contains, among others, a character vector with all implemented languages as well as a data frame with the translations. The main set up of the translation feature is a part of the server function.

As already mentioned, the user can choose the language of the user interface by selecting it using the dropdown menu on the sidebar. The choices available in this menu are given by the languages in the translator object (see the code chunk below).

The setting of the language is then defined in a reactive environment. In this environment, the language which is selected by the user is set as the current language of the application, with English being the default setting when starting the application.

For using the translations for any character string, there needs to be a corresponding entry in the .json file. In the code, this string then has to be put inside the translation function “t()” which is an argument of the translator object created beforehand.

Within this setup, the displayed strings are automatically updated to correspond to the language chosen by the user. This ensures that users speaking different languages can use the application. The .json file can be easily extended to include more languages than English and German.

```
# Server Translation Set up

i18n <- reactive({
  selected <- input$selected_language
  if (length(selected) > 0 && selected %in% translator$languages) {
    translator$set_translation_language(selected)
  }
  translator
})

output$language_selector_input <- renderUI({

  selectInput("selected_language",
              h3(i18n()$t("Language")),
              choices = translator$languages,
              selected = input$selected_language)

})
```

6 Conclusion

In summary, we have succeeded in constructing an application that simplifies the analysis of study progress data. On a user-friendly interface, even non-computer-affine users can examine

study progressions by simply uploading a data set and eventually setting some filters. The visualization of various variables helps to identify correlations or trends.

Above all, the illustration of the cumulative number of ECTS CP acquired is of great benefit. The possibility of filtering the tables makes it possible to analyze on both an individual and a general level. Furthermore, language selection provides a user-friendly environment. We implemented English and German as language options as the University of Duisburg-Essen is a German-speaking university, but also offers further education opportunities with some English language courses for international students and Ph.D. candidates. However, there is also the possibility to add new languages to the program at any time, since the basic framework, as described in chapter 5.3, is already provided. A further suggestion for improving the application would be to not only include comparability for different courses of study (or somewhat different data sets), but also to make different filters of the same data set comparable. This would be useful, for example, if two cohorts were to be compared with each other. The option of uploading multiple files simultaneously would make the application even easier to use.

Software-References

- Attali, D. (2020). *Shinyjs: Easily improve the user experience of your shiny apps in seconds* [R package version 1.1]. R package version 1.1. <https://CRAN.R-project.org/package=shinyjs>
- Chang, W., & Borges Ribeiro, B. (2018). *Shinydashboard: Create dashboards with 'shiny'* [R package version 0.7.1]. R package version 0.7.1. <https://CRAN.R-project.org/package=shinydashboard>
- Chang, W., Cheng, J., Allaire, J., Xie, Y., & McPherson, J. (2020). *Shiny: Web application framework for r* [R package version 1.4.0.2]. R package version 1.4.0.2. <https://CRAN.R-project.org/package=shiny>
- Cheng, J., & Borges, B. (2019). *Pool: Object pooling* [R package version 0.1.4.3]. R package version 0.1.4.3. <https://CRAN.R-project.org/package=pool>
- Henry, L., & Wickham, H. (2019). *Purrr: Functional programming tools* [R package version 0.3.2]. R package version 0.3.2. <https://CRAN.R-project.org/package=purrr>
- Krzeminski, D. (2018). *Shiny.i18n: Shiny applications internationalization* [R package version 0.1.0]. R package version 0.1.0. <https://CRAN.R-project.org/package=shiny.i18n>
- Müller, K., & Wickham, H. (2019). *Tibble: Simple data frames* [R package version 2.1.3]. R package version 2.1.3. <https://CRAN.R-project.org/package=tibble>
- Müller, K., Wickham, H., James, D. A., & Falcon, S. (2020). *Rsqlite: 'sqlite' interface for r* [R package version 2.2.0]. R package version 2.2.0. <https://CRAN.R-project.org/package=RSQLite>
- Perrier, V., Meyer, F., & Granjon, D. (2020). *Shinywidgets: Custom inputs widgets for shiny* [R package version 0.5.1]. R package version 0.5.1. <https://CRAN.R-project.org/package=shinyWidgets>
- R Core Team. (2019). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. Vienna, Austria. <https://www.R-project.org/>
- RStudio Team. (2019). *Rstudio: Integrated development environment for r* [Version 1.2.1541]. Version 1.2.1541. RStudio, Inc. Boston, MA. <http://www.rstudio.com/>
- Ushey, K., Allaire, J., Wickham, H., & Ritchie, G. (2019). *Rstudioapi: Safely access the rstudio api* [R package version 0.10]. R package version 0.10. <https://CRAN.R-project.org/package=rstudioapi>
- Wickham, H. (2016). *Ggplot2: Elegant graphics for data analysis*. Springer-Verlag New York. <https://ggplot2.tidyverse.org>
- Wickham, H. (2019). *Stringr: Simple, consistent wrappers for common string operations* [R package version 1.4.0]. R package version 1.4.0. <https://CRAN.R-project.org/package=stringr>

- Wickham, H. (2020). *Forcats: Tools for working with categorical variables (factors)* [R package version 0.5.0]. R package version 0.5.0. <https://CRAN.R-project.org/package=forcats>
- Wickham, H., François, R., Henry, L., & Müller, K. (2019). *Dplyr: A grammar of data manipulation* [R package version 0.8.0.1]. R package version 0.8.0.1. <https://CRAN.R-project.org/package=dplyr>
- Wickham, H., & Henry, L. (2019). *Tidyr: Easily tidy data with 'spread()' and 'gather()' functions* [R package version 0.8.3]. R package version 0.8.3. <https://CRAN.R-project.org/package=tidyr>
- Wilke, C. O. (2019). *Cowplot: Streamlined plot theme and plot annotations for 'ggplot2'* [R package version 1.0.0]. R package version 1.0.0. <https://CRAN.R-project.org/package=cowplot>
- Xie, Y., Cheng, J., & Tan, X. (2020). *Dt: A wrapper of the javascript library 'datatables'* [R package version 0.13]. R package version 0.13. <https://CRAN.R-project.org/package=DT>

A Appendix

A.1 Description of relevant Variables

Table A1: Description of relevant variables

Variable	Dataset	Description
Student.Pseudonym	all	Student's immatriculation number
Semester	FS Data	Academic semester. The first four digits stand for the year, the fifth digit is either a 1 (summer semester) or a 2 (winter semester). Example: "20062" describes the winter semester 2006/2007, "20071" the summer semester 2007
FS	FS, Pruefung Data	Study semester
Status	FS Data	Status as a student. "R" stands for re-enrollment, "N" for new enrollment, "E" for initial enrollment and "B" for leave of absence
Status.dpp	Pruefung Data	See variable Status
Austrittsgrund	FS, Studium Data	Reason for dropping out
SGCode	all	Code for study program. The last digit represents the examination regulations (PO). A change to the PO is thus considered as a new program in the system
Abschluss_Bezeichnung	FS, Pruefung Data	Name of the degree that students receive upon successful completion of their studies
Fach_Bezeichnung	FS, Pruefung Data	Name of the course of studies
PO	all	Examination regulations
PolyvalentePruefungsnummer	Pruefung Data	Polyvalent examination number
Bezeichnung	Pruefung Data	Name of the examination
Pruefungssemester	Pruefung Data	Semester in which the student was registered for the exam. Structure analogous to the variable 'Semester' in FS Data

Continued on the next page.

Table A1: Description of relevant variables (continued)

Variable	Dataset	Description
Status.Pruefung	Pruefung Data	Status of the exam. "BE" means Passed, "ZU" means Withdrawn, "NB" means Not Passed and "PV" means Examination Existing (applications or performances are available, but the whole module is not yet completed)
Versuchszahl	Pruefung Data	Trial number
Note	Pruefung Data	Grade
VerbuchteECTSCP	Pruefung Data	Received ECTSCP
Start_Semester	Studium Data	Semester in which the studies were started. Structure analogous to the variable 'Semester' in FS Data
Ende_Semester	Studium Data	Semester in which the studies were finished. Structure analogous to the variable 'Semester' in FS Data

End of table.